

Backend Planning Notes

本文件記錄 ICU 護理分配決策支援系統的后端資料庫、API、前後端串接狀態與開發進度。

Current Status

- Backend: Node.js HTTP server + PostgreSQL + REST JSON API.
- Runtime repository: `backend/src/pgRepository.mjs` .
- Database helper: `backend/src/db.mjs` .
- Local DB setup: `npm run db:setup` .
- API server: `npm run api:dev` .
- API base path: `/api/v1` .
- Frontend API client: `src/api/client.ts` .
- Frontend default API URL: `http://127.0.0.1:8787/api/v1` .

Response format:

```
{
  "data": {},
  "meta": {}
}
```

Error format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "欄位格式錯誤",
    "details": {}
  }
}
```

Development Progress

- ☒ Backend planning notes recorded.
- ☒ Step 1: core tables for `shifts` , `nurses` , `beds` , `patients` , and `admissions` .
- ☒ Step 1 API: current shift, nurse list, patient/admission overview.
- ☒ Step 2: burden assessments and nurse TO-DO schema/API.
- ☒ Step 3: allocation runs and allocation items schema/API.
- ☒ PostgreSQL runtime connection.
- ☒ DB setup script for migrations and demo seeds.
- ☒ Frontend API client.
- ☒ Frontend pages/buttons wired to backend APIs.
- ☒ War-room and handoff-sheet read APIs.
- ☐ Step 4: order import.

- ☐ Step 4: persisted handoff snapshots and handoff rows.
- ☐ Production auth / permission model.
- ☐ Automated end-to-end tests.

Implementation Log

2026-05-09 - Step 1 API Implementation

- Added `backend/server.mjs` , a Node HTTP API server.
- Added temporary Step 1 in-memory data/repository.
- Added `npm run api:dev` .
- Implemented CORS headers for local frontend development.
- Implemented API response and error shape.
- Implemented `/health` , `/me` , `/shifts/current` , `/nurses` , `/admissions` , `/nurse/overview` .
- Verified build and lint.

2026-05-10 - Step 2 Burden Assessments And Nurse TO-DO

- Added `backend/db/migrations/002_burden_tasks_schema.sql` .
- Added `backend/db/seeds/002_demo_burden_tasks_data.sql` .
- Added temporary Step 2 in-memory data/repository.
- Implemented:
 - `GET /api/v1/burden-assessments`
 - `PATCH /api/v1/burden-assessments/{assessmentId}`
 - `GET /api/v1/tasks`
 - `PATCH /api/v1/tasks/{taskId}`
- Verified burden and task APIs with local requests.
- Verified build and lint.

2026-05-11 - Step 3 Allocation Runs And Allocation Items

- Added `backend/db/migrations/003_allocation_schema.sql` .
- Added `backend/db/seeds/003_demo_allocation_data.sql` .
- Added temporary Step 3 in-memory data/repository.
- Implemented:
 - `POST /api/v1/allocation-runs/suggest`
 - `GET /api/v1/allocation-runs/{allocationRunId}`
 - `PUT /api/v1/allocation-runs/{allocationRunId}/items`
 - `POST /api/v1/allocation-runs/{allocationRunId}/confirm`
- Suggestion algorithm uses `demo-greedy-v1` : sort patients by burden score descending, then assign each patient to the currently lowest-load nurse.

2026-05-11 - PostgreSQL Runtime Wiring

- Added `pg` dependency.
- Added `backend/src/db.mjs` .
- Added `backend/scripts/setupDatabase.mjs` .
- Added `npm run db:setup` .

- Replaced API runtime imports with `backend/src/pgRepository.mjs` .
- `npm run db:setup` now creates/resets `sad_frontend_v2` , applies migrations, applies seeds, and fills complete demo data.
- Added PostgreSQL-backed implementations for:
 - current user / shift
 - nurses
 - admissions
 - nurse overview
 - burden assessments
 - tasks
 - allocation suggestions / updates / confirmation
 - war-room
 - handoff sheet
- Fixed PostgreSQL status parameter casts for burden/task updates.
- Fixed `burden_values` null handling: when a subjective factor is `null` , the backend deletes that value row instead of inserting an empty row.

2026-05-11 - Frontend Full-Stack Wiring

- Added `src/api/client.ts` .
- Wired:
 - `/nurse/overview`
 - `/nurse/burden-form`
 - `/nurse/todo`
 - `/leader/allocation`
 - `/leader/allocation-result`
 - `/leader/war-room`
- Buttons now connected:
 - burden form 儲存 , 儲存草稿 , 送出
 - TO-DO checkbox
 - allocation 系統建議分床
 - allocation nurse reassignment select
 - allocation 確認送出
 - war-room expand/collapse remains local UI state

Local Runbook

Initial setup:

```
npm install
npm run db:setup
```

Run backend:

```
npm run api:dev
```

Run frontend in another terminal:

```
npm run dev
```

If frontend shows `Failed to fetch`, check that the API is running:

```
curl http://127.0.0.1:8787/api/v1/health
```

If API runs on another port:

```
VITE_API_BASE_URL=http://127.0.0.1:8791/api/v1 npm run dev
```

Verification:

```
npm run db:setup
npm run lint
npm run build
```

Database Domains

- Shift management: `users`, `nurses`, `shifts`, `shift_nurses`.
- Patient and bed management: `beds`, `patients`, `admissions`.
- Burden assessment: `burden_factors`, `burden_assessments`, `burden_values`.
- Tasks: `tasks`.
- Allocation: `allocation_runs`, `allocation_items`.
- Future order import: `order_import_batches`, `orders`.
- Future handoff snapshots: `handoff_snapshots`, `handoff_rows`.

Implemented Endpoints

Core:

- `GET /api/v1/health`
- `GET /api/v1/me`
- `GET /api/v1/shifts/current`
- `GET /api/v1/nurses?shiftId={shiftId}`
- `GET /api/v1/admissions?shiftId={shiftId}&status=active`
- `GET /api/v1/nurse/overview?shiftId={shiftId}`

Burden and tasks:

- `GET /api/v1/burden-assessments?shiftId={shiftId}&scope=mine`
- `PATCH /api/v1/burden-assessments/{assessmentId}`
- `GET /api/v1/tasks?shiftId={shiftId}&assignee=me&status=pending&kind=給藥`
- `PATCH /api/v1/tasks/{taskId}`

Allocation and operational views:

- `POST /api/v1/allocation-runs/suggest`
- `GET /api/v1/allocation-runs/{allocationRunId}`
- `PUT /api/v1/allocation-runs/{allocationRunId}/items`
- `POST /api/v1/allocation-runs/{allocationRunId}/confirm`
- `GET /api/v1/war-room?shiftId={shiftId}`
- `GET /api/v1/handoff-sheets?shiftId={shiftId}`

Burden Factor Codes

Objective factors:

```
[
  "negativePressureIsolation",
  "highVentilatorDemand",
  "medicationTypeCount",
  "medicationFrequency",
  "crrtContinuousA",
  "iabpContinuousB",
  "ecmoContinuousB",
  "proneContinuousB",
  "hypothermiaContinuousB",
  "massiveTransfusionSingleC",
  "plasmaSingleC"
]
```

Subjective factors:

```
[
  "rassScore",
  "agitatedFallRisk",
  "agitatedTubeRemovalRisk",
  "drainageTube",
  "tubeFeeding",
  "dressingChangeFrequency",
  "vitalMonitoringFrequency"
]
```

Subjective scoring:

- RASS `null` : no stored value and 0 points.
- RASS absolute value `0-1` : 0 points.
- RASS absolute value `2-3` : 1 point.
- RASS absolute value `4+` : 2 points.
- Boolean factors: `true` = 2 , `false` = 0 .
- Level factors: low `0` , medium `1` , high `2` .

Patient-Centric Data Preparation

For real data preparation, use a patient-centric JSON shape containing:

- `patient` : `patients`
- `bed` : `beds`
- `admission` : `admissions`
- `shiftContext` : current shift context
- `currentAssignment` : current nurse assignment
- `burdenAssessment` : assessment plus objective/subjective values
- `tasks` : task list
- `allocationSnapshot` : optional current allocation item

The current backend does not yet provide a bulk JSON import endpoint. Real-data import is a recommended next step before Step 4 order import.

Remaining Work

1. Add patient-centric JSON import API for real data preparation.
2. Implement Step 4 order import.
3. Add persisted handoff snapshots and handoff rows.
4. Add auth/session model and role checks.
5. Add automated API and frontend E2E tests.